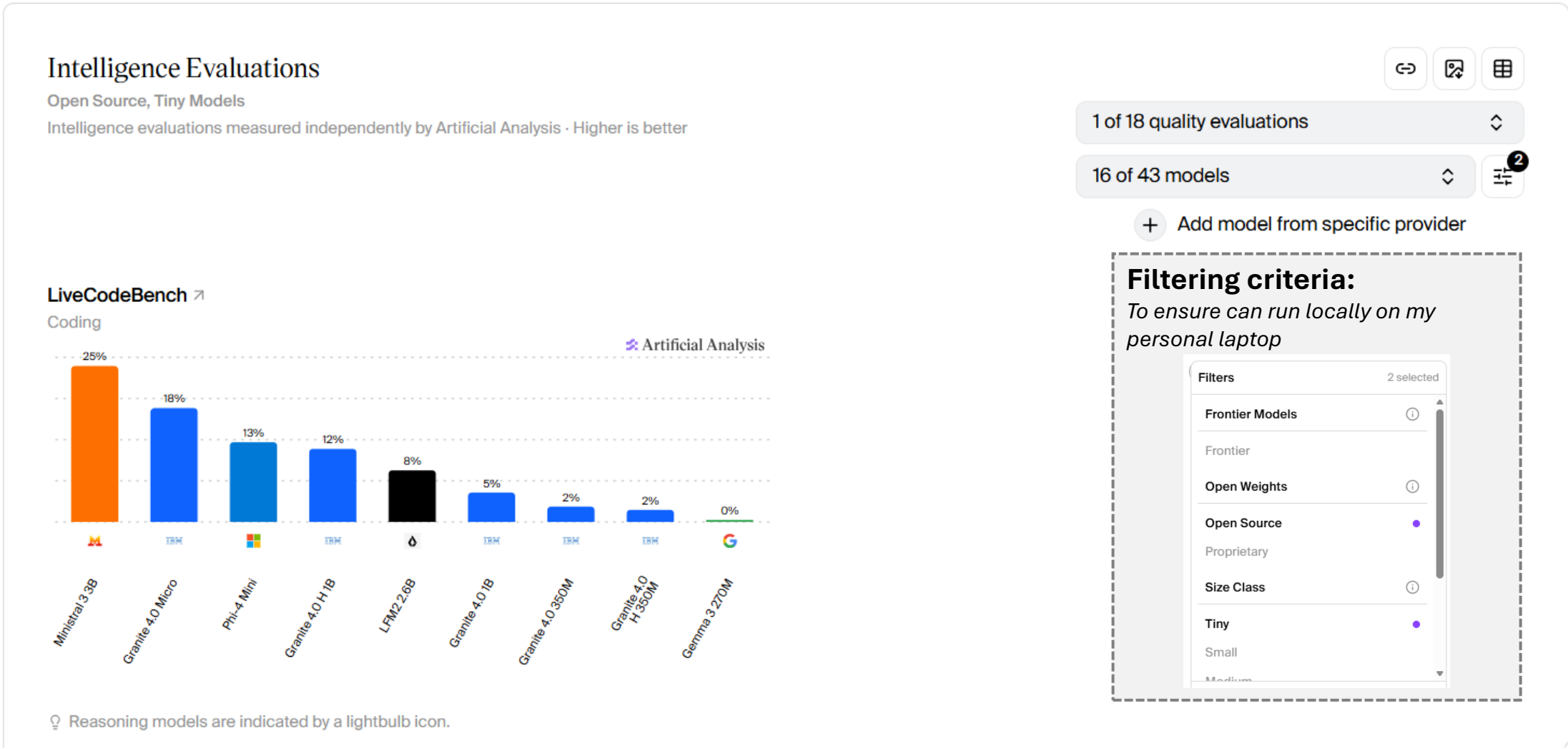


1. Select LLM candidates from Artificial Analysis website

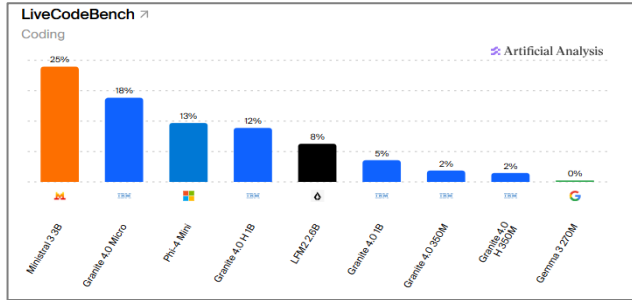
■ Intelligence Breakdown



Source: <https://artificialanalysis.ai/>

2. Build pipeline to test the model candidates

a Sample top 3 open-source & tiny models from LiveCodeBench test result



Source: <https://artificialanalysis.ai/>

Sample models to install (`ollama pull`)

1. minstral-3:3b (3.0 GB)
2. granite4:micro (2.1 GB)
3. phi4-mini (2.5 GB)

Total memory used: 7.6 GB

Source: <https://ollama.com/library/>

Note: 2 frontier models (gpt-5 and gpt-5-nano) will also be included as benchmarks

b Build LLM pipeline to define code generation task

Models & clients dictionary

```
models = ["gpt-5", "gpt-5-nano", "minstral-3:3b", "granite4:micro", "phi4-mini"]
```

```
clients = {"gpt-5": openai, "gpt-5-nano": openai, "minstral-3:3b": ollama, "granite4:micro": ollama, "phi4-mini": ollama}
```

System & prompt and port function

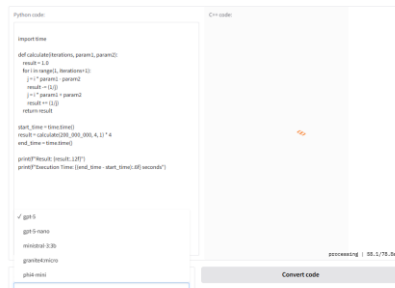
```
def port(model, python):
    client = clients[model]
    response = client.chat.completions.create(model=model, messages=messages_for(python))
    reply = response.choices[0].message.content
    reply = reply.replace('```cpp', '').replace('```', '')
    return reply
```

Python code example to optimize (Pi number calculation)

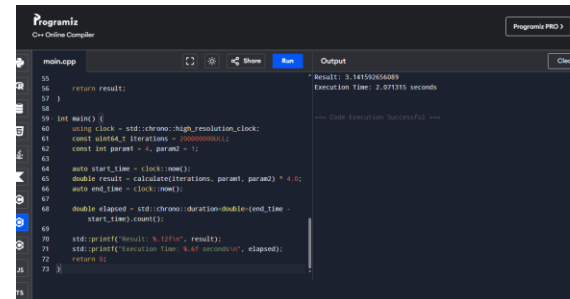
```
(1/2)
pi = ""
import time
def calculate(iterations, param1, param2):
    result = 1.0
    for i in range(1, iterations+1):
        j = i * param1 - param2
        result -= (1/j)
        j = i * param1 + param2
        result += (1/j)
    return result
start_time = time.time()
result = calculate(200_000_000, 4, 1) * 4
end_time = time.time()
print(f"Result: {result:.12f}")
print(f"Execution Time: {(end_time - start_time):.6f} seconds")
"""
(2/2)
```

c Wrap with Gradio UI for easier testing, compile in programiz.com

Gradio UI (able to switch models in dropdown)



Compile in Programiz online C++ compiler



Measure the new code performance

Result: 3.141592656089
Execution Time: 2.071315 seconds

=== Code Execution Successful ===

3. Code Performance Comparison and Leaderboards

No	Category	Models	Pi Calculation Result	Execution Time (in seconds)	Faster by-X	Final Ranks
0	-	(Baseline, python code)	3.141592656089	38.451193	-	-
Results after converted to C++						
1	Closed Source	gpt-5	3.141592656089	1.645104	23.37x faster	2
2		gpt-5-nano	3.272967991442	1.404686268	27.37x faster <i>* with inaccurate result</i>	1
3	Open Source	ministral-3:3b	FAILED	FAILED	-	-
4		granite4:micro	FAILED	FAILED	-	-
5		phi4-mini	FAILED	FAILED	-	-